AI-POWERED SYSTEM

Resume AI

AI Embeddings & ChromaDB

Semantic candidate matching using Vector Embeddings, & ChromaDB vector store

- Jayasri P

Project Overview

What is Resume AI?

Resume AI is a full-stack, AI-powered document search and candidate matching system that leverages Vector Embeddings and Natural Language Processing (NLP) to automate resume screening against job descriptions.

Resume Upload

PDF parsing & OCR text extraction

AI Embedding

sentence-transformers vector encoding

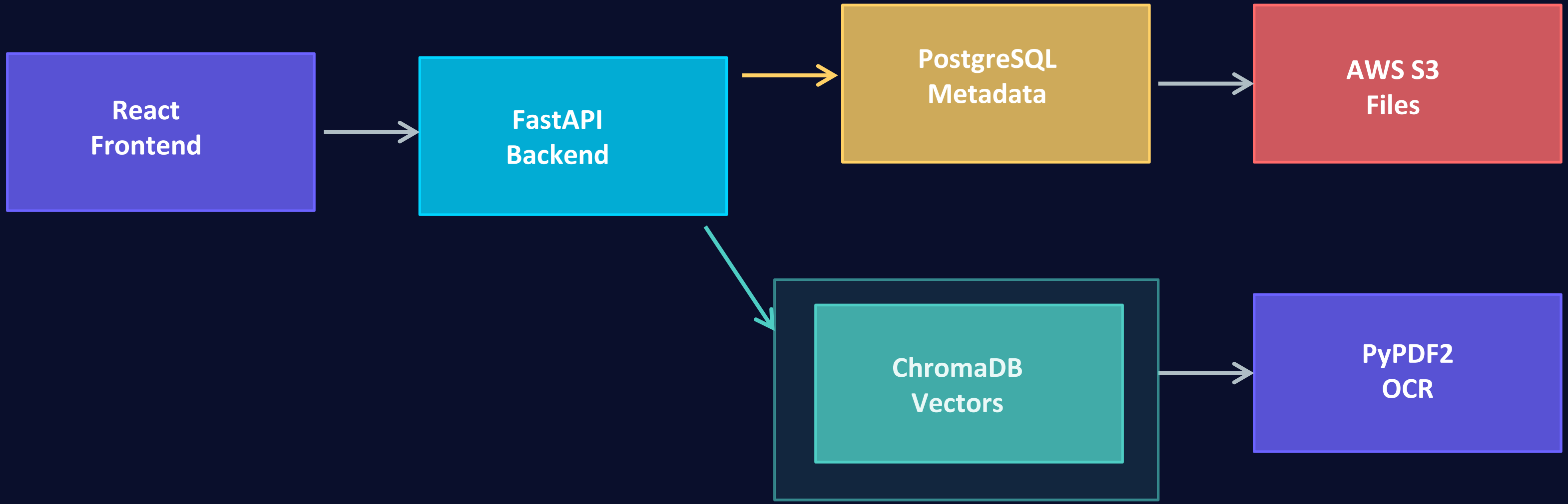
Smart Search

ChromaDB semantic nearest-neighbor

Cloud Deploy

AWS S3(store) + EC2 Docker containers

System Architecture



★ ChromaDB is the AI brain — it stores and searches vector embeddings for semantic matching

Docker | Nginx | sentence-transformers | boto3 | psycopg2

What Are Vector Embeddings?

The core AI concept powering semantic search

Definition

A vector embedding is a numerical representation of text that captures its semantic meaning. Similar texts produce vectors that are mathematically close in high-dimensional space.

Example Vector

"Python developer with ML skills"

```
→  [0.23, -0.87, 0.45, 0.11,
      -0.63, 0.92, ... 384 dims]
```

1

Text Input

Raw resume text or job description fed as input

2

Tokenization

Text split into tokens (words/subwords) by the model

3

Neural Encoding

all-MiniLM-L6-v2 model processes tokens through 6 transformer layers

4

384-dim Vector

Output: a 384-dimensional float array representing semantic meaning

💡 Key Insight: Unlike keyword matching, embeddings understand meaning — "ML engineer" and "machine learning developer" produce similar vectors!

The Model: all-MiniLM-L6-v2

sentence-transformers library — lightweight yet powerful

Model Specifications

Architecture:	MiniLM (distilled BERT)
Layers:	6 Transformer layers
Output Dim:	384 dimensions
Input Limit:	256 tokens (~500 words)
Model Size:	~80 MB (fast loading)
Training Data:	1B+ sentence pairs
Task:	Semantic Similarity
License:	Apache 2.0 (free use)

```python
# Python code in backend
from sentence_transformers import SentenceTransformer

model = SentenceTransformer(
    "all-MiniLM-L6-v2"
)

# Encode a resume
embedding = model.encode(resume_text)
# Returns: numpy array [384]
# Shape: (384,)
```

Why MiniLM? Small size + fast inference + high semantic accuracy = perfect for real-time resume matching

ChromaDB: The Vector Store

High-performance vector database for AI-native applications

What is ChromaDB?

ChromaDB is an open-source, AI-native vector database. It stores high-dimensional embeddings and enables ultrafast similarity search — the backbone of Resume AI's matching engine.

Key Features

- Persistent vector storage
- HNSW index for fast ANN search
- Metadata filtering alongside vectors
- Python-native, no extra infra needed

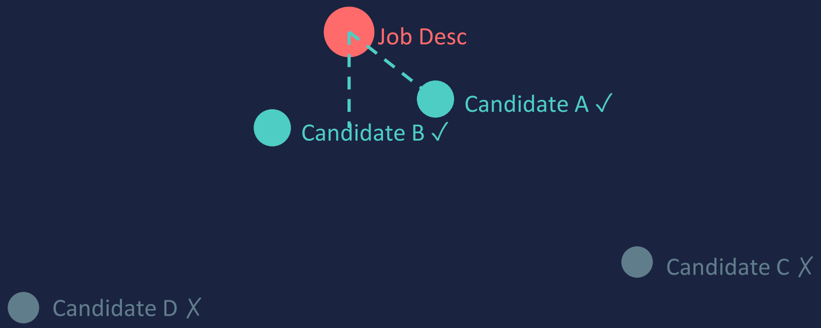
How ChromaDB Works in Resume AI

Resume Uploaded → Text Embedding → Store in ChromaDB → Job Description Vector → Similarity Search

Similarity Search & Scoring

How Resume AI ranks candidates against job descriptions

Vector Space (Conceptual)



Scoring Algorithm

Semantic Score (60%)
from vector cosine distance

Skill Score (40%)
extracted keyword overlap

Final Score = 60% × Semantic + 40% × Skill

```
# ChromaDB query
Cosine similarity measures angle between vectors in 384-dim space
```

ChromaDB returns results with distances — lower distance = higher semantic similarity

Resume Upload & Indexing Pipeline

From PDF upload to searchable vector — step by step

1. **PDF Upload** — User uploads resume via React UI
2. **OCR Extraction** — PyPDF2 extracts raw text content
3. **Skill Extraction** — skills.py scans for keywords
4. **S3 + PostgreSQL** — File + metadata persisted to cloud
5. **ChromaDB Index** — Vector embedding stored & indexed

ChromaDB Role: Step 5 is where the AI magic lives. ChromaDB stores each resume as a 384-dim vector. When a job description is later searched, ChromaDB finds the nearest vectors in milliseconds using its HNSW (Hierarchical Navigable Small World) index.

Complete Tech Stack

Every component and its role in the system

Frontend

- React.js (Vite)
- Vanilla CSS
- Axios HTTP
- React Hooks

Backend

- FastAPI (Python)
- Uvicorn ASGI
- psycopg2
- boto3

AI / ML

- sentence-transformers
- all-MiniLM-L6-v2
- ChromaDB
- PyPDF2 OCR

Data / Cloud

- PostgreSQL
- AWS S3
- AWS EC2
- Docker

Deployment

- Docker Compose
- Nginx (reverse proxy)
- GitHub (CI/CD)
- EC2 (production)
- docker-compose up --build -d

Key Takeaways

Embeddings = Meaning

Vector embeddings capture semantic meaning, not just keywords — enabling true AI-powered search

MiniLM is Efficient

all-MiniLM-L6-v2 delivers high-quality 384-dim embeddings with fast inference and small footprint

ChromaDB Scales

HNSW-indexed vector store enables millisecond similarity queries even with thousands of resumes

Hybrid Scoring Works

Combining semantic (60%) + skill overlap (40%) gives a robust, explainable match percentage

Production-Ready Stack

Docker + AWS + Nginx makes the system deployable, scalable and maintainable in real environments